

TripMate

AI Trip Planner & Map Integration

Phase 6 & Phase 7 — Detailed Implementation Report

Project	TripMate — AI Powered Collaborative Trip Planner
Phase 6	AI Trip Planner (Google Gemini) — ✔ Completed
Phase 7	Map Integration (Geoapify + Leaflet) — ✔ Completed

1. Executive Summary

TripMate is a collaborative trip-planning platform. Two major phases extend it from a static itinerary tool into an **intelligent, geospatial-aware planning assistant**:

Phase	What it adds	Core technology
Phase 6 — AI Trip Planner	Auto-generates a structured, day-wise itinerary from trip details	Google Gemini API (gemini-2.5-flash)
Phase 7 — Map Integration	Interactive map with destination view and category-based nearby search	Geoapify APIs + Leaflet

Core idea: Phase 6 answers "what should I do?" — Phase 7 answers "where is it?". Combined, they turn a list of AI suggestions into something the user can actually navigate to.

2. Technology Stack

Layer	Phase 6	Phase 7
Primary service	Google Gemini API	Geoapify (Geocoding, Places, Map Tiles)
Database	MongoDB	MongoDB
Frontend	React	React + react-leaflet
Backend	Node.js + Express	Node.js + Express

Both phases share one MERN-style stack — only the external service integration differs, and both are wired into the app the same way: through an authenticated Express controller that talks to the third-party API on the server side.

3. Shared Architecture Pattern — The Backend Proxy

Before diving into each phase individually, it's worth understanding the one architectural decision both phases share, because it explains almost every design choice that follows.

KEY CONCEPT
Never call a paid/keyed third-party API directly from the frontend. If React called Gemini or Geoapify directly, the API key would have to live in client-side JavaScript — visible to anyone who opens the browser's dev tools. Instead, both phases route requests through an Express controller:

React (axios) → Express Controller (holds the API key) → Third-Party API (Gemini / Geoapify)

This single pattern gives three benefits at once:

- **Security** — the API key never leaves the server.
- **Control** — the backend can authenticate the user, validate input, and shape/trim the response before sending it to the client.
- **Cost management** — the backend is the natural place to add caching or rate-limiting in front of a paid API.

4. Phase 6 — AI Trip Planner

4.1 Request Flow

- 1 User clicks **Generate Itinerary** → frontend calls `POST /api/ai/trips/:id/itinerary`
- 2 `middleware/auth.js` checks the session token and confirms the user owns this trip
- 3 `Trip.findById()` loads trip data — destination, dates, budget, travelers
- 4 `buildPrompt()` assembles a prompt that demands a strict JSON response
- 5 `generateItinerary()` sends the prompt to Gemini (gemini-2.5-flash) via the `@google/genai` SDK
- 6 The response text is cleaned (e.g. stripped of markdown fences) and passed to `JSON.parse()`
- 7 The parsed object is saved onto the trip document with `trip.save()` — this is the cache
- 8 Frontend calls `setTrip()`; `TripDetails.jsx` re-renders the itinerary UI
- 9 Future visits read the cached itinerary directly from MongoDB — Gemini is called again only on **Regenerate**

4.2 Key Concept — Prompt Engineering

KEY CONCEPT

The entire feature's reliability rests on one function: `buildPrompt()`. A loosely written prompt lets Gemini reply with conversational text ("Sure! Here's your itinerary:") wrapped around the JSON — and that extra text breaks `JSON.parse()`. The fix is to make the prompt a strict contract: state the exact JSON shape expected, and instruct the model to return *only* that JSON, nothing else.

Illustrative prompt structure (not exact wording):

```
Generate a day-wise itinerary for a trip to {destination}
from {startDate} to {endDate} for {travelers} traveler(s),
budget {budget}. Respond with ONLY valid JSON, no extra text:

{
  "days": [{ "day": 1, "activities": [...], "meals": [...] }],
  "budgetEstimate": { "accommodation": 0, "food": 0, "transport": 0 },
  "packingChecklist": [...],
  "hotelRecommendations": [...],
  "foodRecommendations": [...],
  "travelTips": [...],
  "safetyRecommendations": [...]
}
```

Why gemini-2.5-flash specifically: Gemini offers multiple model tiers with different speed/quality trade-offs. The "flash" variant is optimized for low latency rather than maximum reasoning depth — a good fit here because the user is watching a loading spinner in real time, waiting for a response, rather than this being a background batch job where a slower, more thorough model would be justified.

4.3 Generated Itinerary — What's Inside

Section	Description
Day-wise plan	Morning / afternoon / evening activities for each day of the trip
Budget estimate	Approximate cost breakdown across categories
Packing checklist	Items suited to the destination and expected weather
Food recommendations	Local cuisine and must-try dishes
Hotel recommendations	Budget-appropriate stay options
Travel tips	Local customs and transport advice
Safety recommendations	Area-specific safety notes

4.4 Key Concept — Caching Strategy

KEY CONCEPT

The itinerary is stored **embedded inside the Trip document** in MongoDB, rather than in a separate collection. This is a deliberate schema choice: because an itinerary always belongs to exactly one trip and is always read together with the trip's other details, embedding means a single `Trip.findById()` call returns everything needed — no extra join or second query. If itineraries were their own collection, every page load would need two round-trips to the database instead of one.

- Every Gemini call consumes API quota and has a real cost — caching means a trip's itinerary is generated *once*, not on every page visit.
- "Regenerate" is opt-in only, so cost stays predictable — the first successful generation is treated as the source of truth until the user explicitly asks for a new one.
- Caching also improves perceived performance: a cached itinerary loads instantly, while a fresh AI generation can take a few seconds.

4.5 Error Handling

- If Gemini is unreachable or returns an unexpected format, the backend catches the failure and returns a clean error response instead of crashing the request.
- `JSON.parse()` is wrapped in a try/catch — a parse failure triggers a defined fallback rather than an unhandled exception reaching the frontend.

4.6 Modules & Core Functions

BACKEND	FRONTEND
• <code>controllers/aiController.js</code> — prompt building + Gemini call logic • <code>routes/aiRoutes.js</code> — route definitions • <code>models/Trip.js</code> — schema; stores the itinerary field • <code>middleware/auth.js</code> — ownership/auth check	• <code>pages/TripDetails.jsx</code> — itinerary display, Generate/Regenerate button • <code>api/axios.js</code> — shared axios instance

Function	Role
buildPrompt()	Builds the structured prompt sent to Gemini
generateItinerary()	Calls the Gemini API and returns the raw response
Trip.findById()	Fetches the trip (and its cached itinerary) from MongoDB
trip.save()	Persists the newly generated itinerary
JSON.parse() / JSON.stringify()	Converts between Gemini's text response and JS objects
api.post()	Frontend → backend request to trigger generation
setTrip()	Updates React state to reflect the new itinerary

5. Phase 7 — Map Integration

5.1 Request Flow

- 1 User opens the **Map** tab inside TripDetails.jsx, mounting TripMap.jsx
- 2 The destination string is **geocoded** into lat/lon coordinates via the backend proxy
- 3 MapContainer centers on those coordinates; a destination Marker is placed
- 4 User selects a category (e.g. Hotels) → getNearbyPlaces() runs
- 5 api.get('/api/maps/nearby', { lat, lon, category }) calls the backend
- 6 mapController.js attaches the Geoapify key server-side and calls the **Places API**
- 7 Results (name, coordinates, address) are returned to the frontend
- 8 setMarkers() updates state → one Marker + Popup renders per result
- 9 Separately, navigator.geolocation() adds a "you are here" marker, if permission is granted

Example proxy request/response:

```
GET /api/maps/nearby?lat=32.2432&lon=77.1892&category=hotel

Response:
{
  "places": [
    { "name": "Example Hotel", "lat": 32.244, "lon": 77.190, "address": "..."},
    { "name": "Another Stay", "lat": 32.241, "lon": 77.187, "address": "..."}
  ]
}
```

The frontend never sees or transmits the Geoapify API key — only the backend holds it, which is the entire point of the proxy pattern described in Section 3.

5.2 Geoapify Services Used

Service	Purpose	Notes
Geocoding API	Converts a destination name into lat/lon coordinates	Always runs first — nothing can be plotted without coordinates
Places API	Category-based nearby search around given coordinates	Called once per category the user selects
Map Tiles	Supplies the visual map background	Rendered by Leaflet's TileLayer

5.3 Key Concept — Leaflet vs. a JS-heavy Maps SDK

KEY CONCEPT

Leaflet was chosen as a lightweight, open-source mapping library rather than a heavier proprietary maps SDK. It renders markers, popups, and tile layers with a small footprint and pairs naturally with react-leaflet, which wraps Leaflet's imperative API in declarative React components (<MapContainer>, <Marker>, etc.) — meaning map state can be driven by normal React state (useState) instead of manual DOM manipulation.

5.4 Leaflet / React-Leaflet Building Blocks

Component / Hook	Role
MapContainer	Wraps the entire map; sets initial center and zoom level

TileLayer	Renders the Geoapify map tiles as the visual background
Marker	Pins a specific location — destination, a nearby place, or the user
Popup	Shows extra info (name, address, distance) when a marker is clicked
useMap()	Gives direct access to the underlying map instance (e.g. to recenter programmatically)

5.5 Nearby Place Categories

Category	Use Case
Hospitals	Finding emergency medical help near the destination
Hotels	Stay options around the destination
Restaurants	Food options nearby
Petrol Pumps	Fuel stops, useful for road trips
Tourist Attractions	Sightseeing spots to explore

Each category triggers its own scoped Geoapify query — **all categories are not fetched at once**, which keeps the initial map load fast and limits API usage to what the user actually views.

5.6 Geolocation & Dynamic Markers

- navigator.geolocation() is a browser API that detects the user's current position; if the user denies permission, the map gracefully falls back to a destination-only view.
- setMarkers() combined with useState/useEffect re-renders the map whenever the category changes — no full page reload needed.
- useCallback memoizes fetch functions like getNearbyPlaces() so they aren't recreated on every render, which avoids unnecessary re-renders of the map and marker layers.

5.7 Error Handling

- An invalid destination name (geocoding failure) surfaces a clear, specific error message instead of a silent failure.
- A Geoapify timeout or outage falls back to a defined UI state rather than crashing the map component.

5.8 Modules & Core Functions

BACKEND

- controllers/mapController.js — geocoding + nearby-places logic
- routes/mapRoutes.js — /api/maps/nearby route definition
- server.js — mounts map routes into the Express app

FRONTEND

- components/TripMap.jsx — core Leaflet map logic
- pages/TripDetails.jsx — hosts the Map tab
- api/axios.js — shared axios instance

Function	Role
getNearbyPlaces()	Fetches nearby places via the backend proxy
api.get()	Frontend → backend request
fetch()	Backend → Geoapify API call
navigator.geolocation()	Detects the user's current location
setMarkers()	Updates marker state, triggering a map re-render

6. Cross-Cutting Concerns

6.1 Security

- **API key protection** — both the Gemini and Geoapify keys live only in backend environment variables (.env), never in frontend code or client-visible network requests.
- **Auth middleware** — middleware/auth.js guards itinerary generation so a user can only act on trips they own, preventing other users from triggering (rate-limited, cost-incurring) Gemini calls on someone else's trip.
- **Input validation** — destination strings and coordinates are validated/sanitized before being forwarded to third-party APIs, so malformed or malicious input doesn't reach Gemini or Geoapify unchecked.
- **Usage discipline** — caching (Phase 6) and category-scoped queries (Phase 7) both limit how often paid third-party APIs are actually called.

6.2 Performance

- **Itinerary caching** removes repeat Gemini calls entirely for a trip that hasn't changed — this is the single biggest latency and cost win in the system.
- **On-demand category fetch** means nearby places are only requested for the category currently in view, not every category up front.
- **useCallback** prevents redundant function recreation, which in turn avoids unnecessary re-renders of the map and its marker layers.
- **Viewport-based tile loading** is standard Leaflet behavior — only tiles for the visible area/zoom level are loaded, not the entire world map.

7. Project Structure

```
TripMate/
├── backend/
│   ├── controllers/
│   │   ├── aiController.js      ← Phase 6
│   │   └── mapController.js    ← Phase 7
│   ├── routes/
│   │   ├── aiRoutes.js        ← Phase 6
│   │   └── mapRoutes.js       ← Phase 7
│   ├── models/
│   │   └── Trip.js            ← shared by both phases
│   ├── middleware/
│   │   └── auth.js
│   ├── config/
│   │   └── db.js
│   └── server.js
└── frontend/
    ├── src/
    │   ├── pages/
    │   │   └── TripDetails.jsx  ← hosts both AI + Map tabs
    │   ├── components/
    │   │   └── TripMap.jsx      ← Phase 7
    │   └── api/
    │       └── axios.js         ← shared axios instance
```

TripDetails.jsx and Trip.js are shared by both phases — this is exactly why the two features feel like one connected experience rather than two separate add-ons bolted together.

8. Summary

Phase 6 introduces AI-powered itinerary generation using the **Google Gemini API** — day-wise planning, budget estimation, hotel and food recommendations, a packing checklist, travel tips, and safety notes, all cached in MongoDB and regenerable on demand.

Phase 7 integrates **Geoapify APIs** with **Leaflet** to deliver an interactive map experience — destination visualization, category-scoped nearby-place search, geolocation support, and dynamic marker rendering.

Together, these two phases combine AI-generated planning with real-world geospatial data through a consistent, secure backend-proxy architecture, making TripMate a complete, intelligent, collaborative trip-planning platform.

Status	✔ Phase 6 Completed · ✔ Phase 7 Completed
Project	TripMate — AI Powered Collaborative Trip Planner